An Indoor Mobile Robot (P.C.B) for Corridor Navigation and Secure Delivery Using IMU-Aided Control and ArUco Landmarks

Hatice Nur Kömürcü, Deniz Karabacak, Emre Ertürk, and Tuna İyiarı
Department of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey

***Abstract*—In this paper, we demonstrate the design and iterative development of an indoor mobile robot that performs a simple secure delivery mission between two indoor landmarks. The robot was built through three mechanical prototypes with an explicit focus on reusability, recyclable materials, and a low-budget, cost-effective component selection during rapid iteration. The final system combines (i) reactive corridor navigation using an ultrasonic range sensor mounted on a scanning servo, (ii) IMU-aided straight-line control with a lightweight calibration routine that compensates both gyroscope bias and motor imbalance, and (iii) a vision-based mission trigger using ArUco markers for reliable start/stop at the base and the target. A secured cargo box is integrated with a keypad, LCD status display, and a servo-actuated lock. We report design trade-offs observed during the transition from camera-based learning attempts to a robust behavior-based navigation stack and discuss power-management lessons learned when integrating an embedded GPU platform.**

***Index Terms*—indoor mobile robotics, differential drive, IMU, PID control, reactive navigation, ArUco markers**

I. Introduction

Indoor service robots are increasingly deployed for repetitive transport tasks such as document delivery and small-item logistics. In student-scale projects, the main challenge is typically not a single algorithmic component, but the integration of mechanical constraints, power delivery, sensing, and reliable autonomy under limited compute, limited datasets, and strict budget constraints.

We developed an indoor mobile robot to navigate corridor-like environments, stop at a target location to allow a user to unlock a secure compartment, and then return to a base location. Over two months, the system evolved through multiple prototypes driven by practical constraints: structural rigidity, total mass, turning stability, and power distribution between motors, microcontroller, and an embedded device.

Contributions. The main contributions of this work are:

1. An end-to-end system design that couples low-level real-time control on an Arduino Mega with high-level mission logic on an NVIDIA Jetson Nano.

2. A "smart" calibration sequence that learns gyroscope bias and a motor power bias term to reduce straight-line drift, complemented by a gentle PID heading correction.

3. A robust navigation approach for L-shaped corridors based on obstacle confirmation (time-based) and a right-priority look-around decision using a servo-mounted ultrasonic sensor.

4. A practical discussion of multi-prototype construction emphasizing recyclable/reusable materials and power/weight trade-offs. **5.** Design choices and integration lessons aimed at achieving a cost-effective indoor delivery platform using widely available, low-cost sensors and components.

II. Related Work

Reactive navigation and wall-following behaviors are widely used for robust indoor motion in structured environments, especially when global maps are unavailable or when compute and sensing are constrained. Classic mobile robotics texts discuss behavior-based navigation and control primitives used for tasks such as corridor following and obstacle avoidance [1].

For local obstacle avoidance, algorithms such as the Vector Field Histogram (VFH) demonstrate how range sensing can be transformed into fast steering decisions suitable for real-time platforms [2]. While our robot does not implement VFH, the same design principle motivates our lightweight, range-based policy.

ArUco markers provide a practical compromise between full SLAM and purely reactive navigation. ArUco-type markers are designed for reliable detection and pose estimation under occlusions and varying imaging conditions [3]. Recent studies demonstrate that ArUco landmarks can support indoor localization, docking, and navigation pipelines with modest sensing and compute requirements [4].

Finally, MEMS gyroscopes require calibration and drift mitigation for heading control. Prior work has studied drift reduction and minimal-drift heading estimation for indoor robots using rate gyros [5]. Our approach follows the same philosophy but keeps computation minimal to run on a microcontroller.

III. System Overview

A. Hardware Architecture

The final platform is a differential-drive robot with four DC motors driven by an L298N motor driver. An Arduino Mega 2560 executes the real-time control loop, reads the MPU-6050 IMU (I^2C) and the ultrasonic sensor (trigger/echo), controls two MG995 servos (head scan and cargo lock), and interfaces

with a keypad and a Nokia 5110 (PCD8544) LCD for local user interaction.

An NVIDIA Jetson Nano 2GB Developer Kit runs the high-level mission process, receives camera frames from a Raspberry Pi Camera v2, detects ArUco markers with OpenCV, and sends serial commands to the Arduino over USB. For runtime convenience, the Jetson software stack was executed inside a prebuilt OpenCV-enabled Docker image (commonly used in Jetson ecosystems).[1]

B. Software Partitioning

We intentionally split the autonomy stack:

- **Microcontroller (Arduino)**: deterministic actuation, IMU integration, calibration, PID heading correction, ultrasonic obstacle handling and autonomous movement, and secure box UI/lock logic.
- **Companion computer (Jetson)**: vision processing and mission-level state machine (search/approach/stop/return) using ArUco landmarks and serial commands.

This separation isolates high-frequency control from variable-latency vision workloads.

C. Remote Operation and Live Camera Monitoring

During development and testing, we connected to the Jetson Nano via SSH from a laptop on the same local network to launch and supervise the mission software from a terminal. In parallel, the mission application exposed a lightweight local web server that streamed the live camera feed (MJPEG) to the same network. This enabled real-time visual monitoring and debugging without requiring physical access to the robot's display hardware.

IV. Iterative Prototyping with Recyclable/Reused Materials

We built and evaluated three main prototypes, reusing components as much as possible and prioritizing recyclable materials during early iterations.

A. Prototype 1: Cardboard Chassis for Drive Validation

The first prototype used a cardboard box chassis with DC motors hot-glued to the body and simple wheels. Power was provided by four 1.5 V AA batteries and an L298N driver. The goal was limited to validating basic driving and motor direction control. After confirming mechanical feasibility, we integrated an Arduino Mega 2560 and ran a basic script (e.g., drive-forward for 4 s, stop, repeat) to validate repeatability and wiring.

B. Prototype 2: Two-Layer Layout and Powerbank-Driven Compute

After adding an ultrasonic sensor, an IMU, and building a sensor "hub," we integrated a Raspberry Pi Camera v2 and a Jetson Nano. The increased mass resulted in a second chassis in which motors were placed between two structural layers while compute and sensors were mounted on the top layer. Powering the Jetson Nano via a 20 000 mAh powerbank increased overall weight and increased turning instability, especially under acceleration.

We attempted to simplify power by removing AA cells and feeding the motor driver from the same powerbank. However, current transients from motor loads affected compute stability and also limited effective motor torque. These issues motivated a third prototype and a revised power approach.

[1]In our setup we used a Jetson-oriented OpenCV container image from the `dustynv` ecosystem; equivalent OpenCV builds can also be used.

TABLE I
SUMMARY OF DESIGN ITERATIONS (QUALITATIVE).

Proto	Body/Materials	Key outcome / issue
P1	Cardboard + hot glue	Validated drivetrain and basic motion scripts
P2	Two-layer body	Jetson + powerbank mass; unstable turns; power transients
P3	3D-printed skeleton + lightweight layers	Stable mounting, improved coupling, mission-ready integration

C. Prototype 3: 3D-Printed Skeleton, Lightweight Layers, and Improved Power

The final chassis uses a 3D-printed H-shaped bottom skeleton that provides motor bays and consistent alignment. Wheels were also 3D-printed; due to wear in wheel hubs during testing, we designed and printed D-type 4 mm shaft adapters to ensure tight motor-wheel coupling.

The mid-structure uses a foam (styrofoam) support block for vibration damping and to host the powerbank and Jetson Nano, followed by a lightweight craft-board layer reinforced with thin wooden sticks. The second layer houses the Arduino, sensors, camera, and Li-ion 18650 cells. The head module includes an MG995 servo that rotates the ultrasonic sensor for look-around. The top layer contains the secure cargo box with an external keypad and LCD, and a second MG995 servo that activates the locking mechanism.

V. Low-Level Control and Reactive Navigation

A. Smart Calibration: Gyro Bias and Motor Bias Learning

On startup, the robot performs an 8 s calibration routine:

- **Gyro bias (3 s)**: the MPU-6050 Z-axis gyro output is sampled while stationary to estimate the zero-rate offset.
- **Motor bias (5 s)**: the robot drives open-loop straight for 5 s while integrating yaw drift. The measured yaw is converted into a *motorBias* term that is added to the left PWM and subtracted from the right PWM. This compensates persistent motor mismatch.

The motor bias is bounded to avoid extreme skew and is reset alongside yaw and PID memory before entering mission execution.

B. Heading Stabilization with Gentle PID

During forward motion, a PID controller regulates yaw error $e = \psi - \psi^*$, where ψ is the integrated yaw and ψ^* is the target yaw captured when entering driving mode. Since the learned motor bias already corrects most systematic drift, the

PID gains are intentionally conservative (e.g., $K_p = 12$, $K_i = 0.05$, $K_d = 5$ in our implementation). The resulting correction is combined with the motor bias:

$$u_L = v_0 + b + u_{PID}, \qquad u_R = v_0 - b - u_{PID},$$

with saturation to valid PWM bounds. In case of a large yaw error (e.g., $> 20°$), the robot stops and performs an in-place rotation to re-acquire the target heading.

C. Obstacle Confirmation and Right-Priority Look-Around

Obstacle handling follows a two-stage policy:

Confirmation: if an obstacle is detected within 200 cm, the robot stops and waits for up to 10 s, repeatedly checking if the obstacle clears. This avoids unnecessary detours around transient obstacles (e.g., a person walking across the path).

Decision: if the obstacle persists, the ultrasonic sensor is rotated to the right. If the right-side distance is "clear" (interpreted as ≥ 400 cm or out-of-range), the robot turns right by approximately $90°$; otherwise it turns left by $90°$. This policy enables robust operation in L-shaped corridors without requiring a global map.

During servo movement, yaw integration is frozen to avoid false yaw updates due to vibration. During the actual turn, integration is re-enabled, and turn completion is detected via yaw magnitude with momentum compensation.

VI. Mission Logic with ArUco Landmarks

Purely camera-based end-to-end driving (behavioral cloning) was initially targeted; however, limited dataset size/diversity and constrained memory on the embedded platform prevented reliable training and deployment of a CNN-based model. A teleoperation-based "path learning" approach (keyboard control) improved repeatability but was still not fully autonomous.

As a result, we adopted a hybrid mission design: the Arduino executes local autonomy for corridor motion, while the Jetson uses ArUco landmarks for mission-level triggers. ArUco markers were placed at the base and at the target location. The Jetson runs a vision loop that:

- Detects marker IDs using OpenCV ArUco detection and pose estimation [3].
- When the target marker is detected, commands the robot to approach and stop at a short distance (10 cm threshold in our implementation), then waits for a fixed interval to allow the user to open/close the cargo lock.
- Commands a $180°$ in-place turn and resumes autonomous corridor navigation until the base marker is detected, at which point the robot stops and reports mission completion.

This hybrid approach reduces reliance on large training datasets while still enabling a repeatable mission lifecycle.

In addition to serial command exchange, the Jetson-side software also served the current camera frames over the local network. This live stream was used to verify marker detection quality (lighting, pose, distance) while the robot was controlled and observed remotely via SSH.

VII. Experiments and Observations

We evaluated the final prototype in a corridor-like indoor environment with L-shaped segments. Key qualitative observations include:

1. Stability: the 3D-printed motor bays and wheel adapters reduced mechanical misalignment and improved straight-line repeatability.

2. Power management: separating compute power (powerbank) from high-current motor loads improved system stability and reduced resets under acceleration.

3. Navigation robustness: the obstacle confirmation step avoided false turns when encountering temporary obstacles, while the right-priority look-around produced consistent behavior in L-shaped corridors.

4. Landmark triggering: ArUco-based mission triggering provided a reliable alternative to small CNN datasets and removed the need for complex localization.

VIII. Discussion

Our experience shows that student-scale indoor robotics projects particularly benefit from an iterative mechanical design process. Early-stage cardboard prototypes enable rapid experimentation at low cost, allowing design flaws to be identified quickly, while transitioning to 3D-printed components in later stages improves structural alignment, robustness, and long-term durability. From a system perspective, maintaining a cost-effective design goal helped prioritize reuse of parts across prototypes and prefer readily available off-the-shelf modules. From a control perspective, adopting a hybrid autonomy strategy, where local reactive behaviors are complemented by sparse visual landmarks, proved more effective than purely learning-based approaches, especially under constraints on available data and computational resources. Additionally, prioritizing calibration early in the control pipeline was crucial: a short, automated calibration routine substantially reduced sensor and motion drift, which in turn simplified subsequent control tuning and improved overall system reliability.

Despite these advantages, the system has several limitations. The lack of a global mapping capability restricts long-term navigation and loop closure, while wall-following policies may fail in environments where walls are not continuous or well-defined. Furthermore, the reliability of visual marker detection remains sensitive to variations in lighting conditions and camera calibration quality, which can negatively impact localization and control performance.

IX. Conclusion and Future Work

We presented a recyclable multi-prototype indoor delivery robot that integrates IMU-aided control, reactive obstacle handling, and ArUco-based mission landmarks, along with a servo-locked secure cargo box. The final platform achieved repeatable corridor navigation and mission completion without requiring large training datasets, while keeping the overall approach cost-effective through reusable mechanical iterations and low-cost sensing/actuation modules.

Future work will focus on (i) improved power regulation and battery monitoring, (ii) fusing wheel odometry with IMU for better heading estimation, (iii) more expressive local planning (e.g., histogram-based methods) [2], and (iv) systematic quantitative evaluation (success rate, time-to-goal, energy consumption) across different corridor layouts.

REFERENCES

[1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. MIT Press, 2011.

[2] J. Borenstein and Y. Koren, "The vector field histogram—fast obstacle avoidance for mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.

[3] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.

[4] A. Sampathkrishna, "Aruco marker based localization and node graph approach to mapping," arXiv preprint, 2022.

[5] S. K. Hong and S. Park, "Minimal-drift heading measurement using a MEMS gyro for indoor mobile robots," *Sensors*, vol. 8, no. 11, pp. 7287–7300, 2008.

APPENDIX A
ADDITIONAL RESULTS

A. Test and Progress Video Playlist

The experimental test runs are documented in a YouTube playlist. **Playlist link:** https://youtube.com/playlist?list=PLwDQJAClSxDNaZMpJzPJgvE02jZJuD9hk&si=x1WH1E5mj55TMsnJ

B. Project Photographs

This appendix includes photographs from the iterative prototyping process and the final integrated platform.



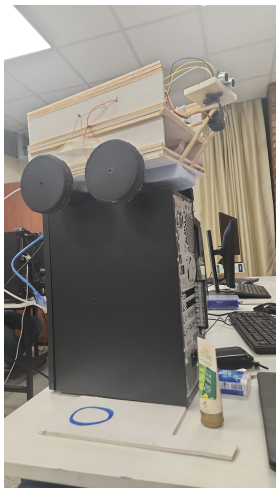
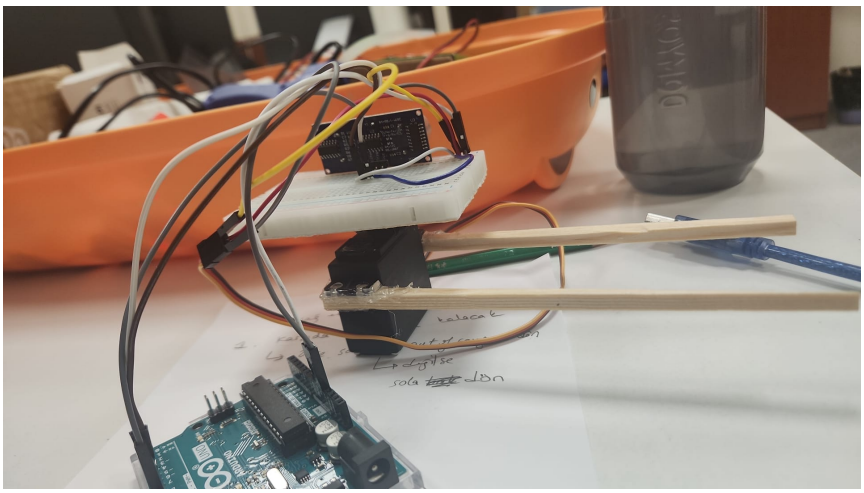


Fig. 1. Prototype and system photographs (set 1).



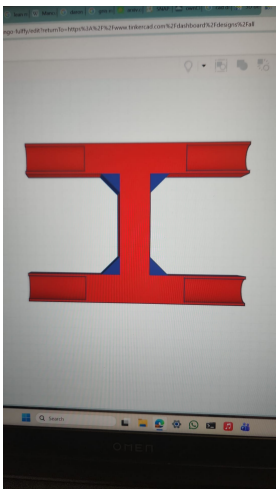


Fig. 2. Prototype and system photographs (set 2).

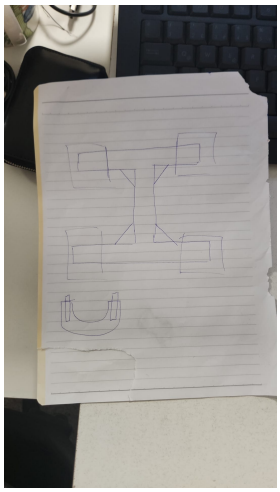




Fig. 3. Prototype and system photographs (set 3).



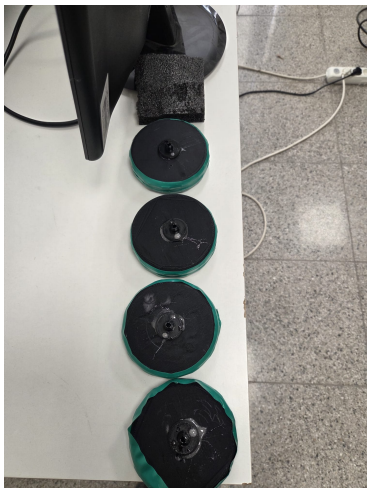
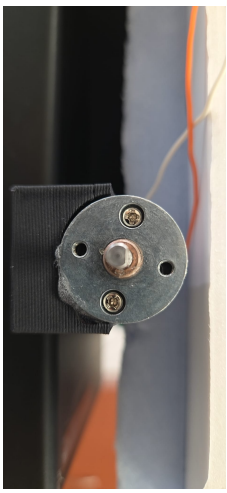


Fig. 4. Prototype and system photographs (set 4).



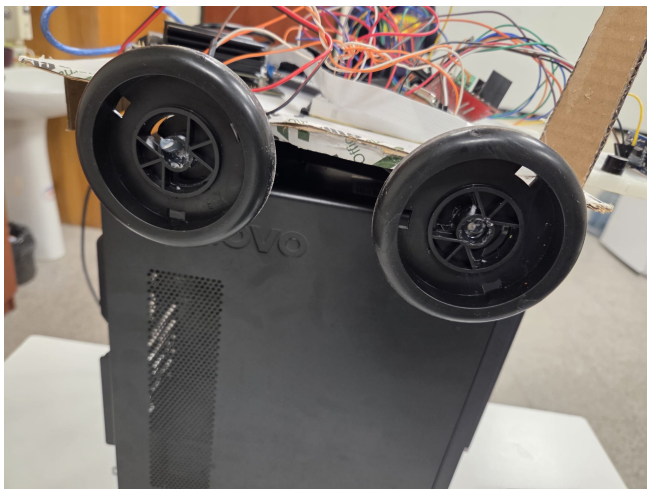


Fig. 5. Prototype and system photographs (set 5).



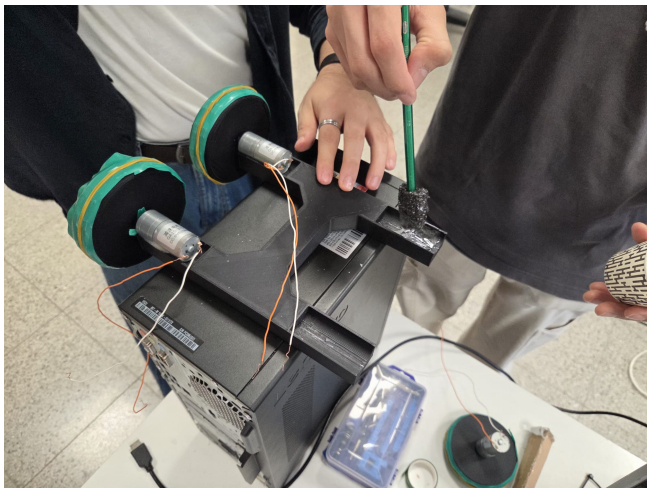
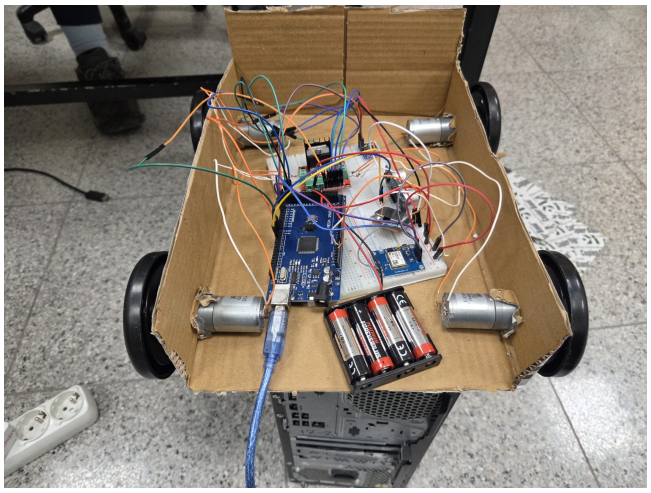


Fig. 6. Prototype and system photographs (set 6).



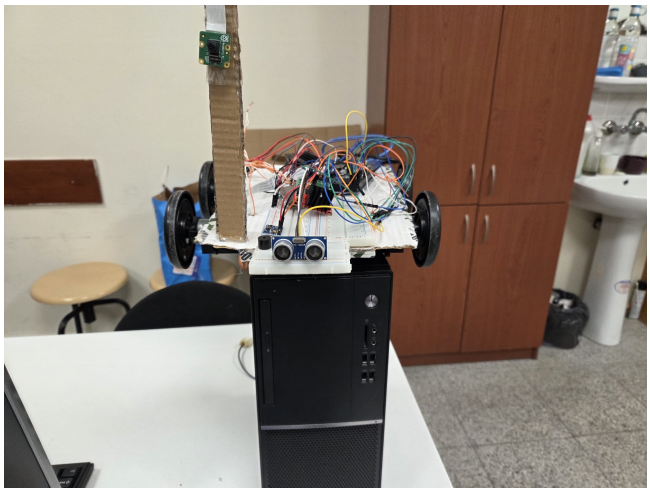
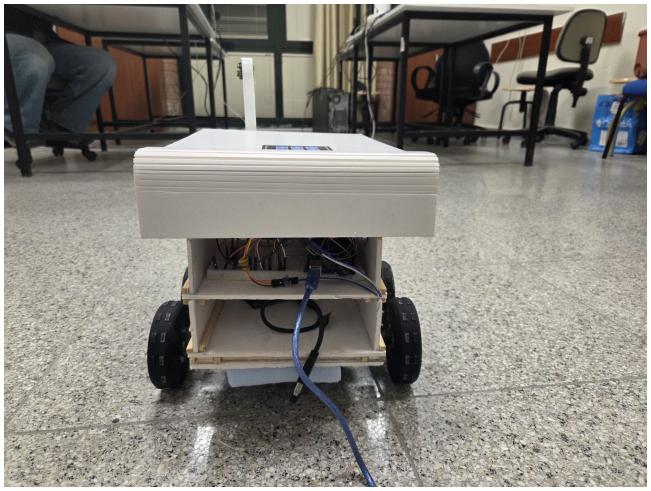


Fig. 7. Prototype and system photographs (set 7).



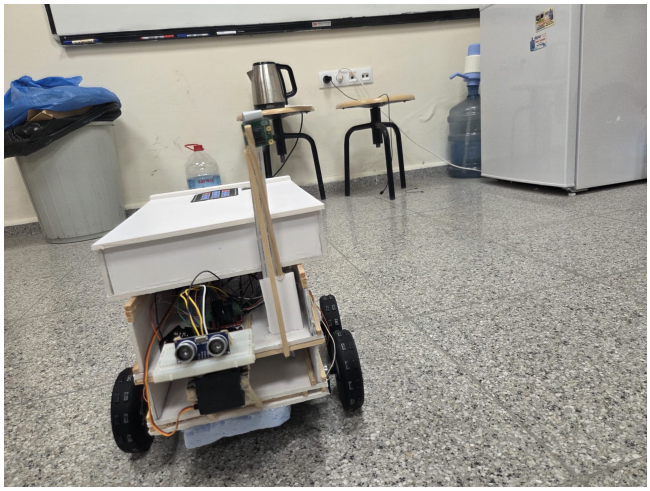


Fig. 8. Prototype and system photographs (set 8).

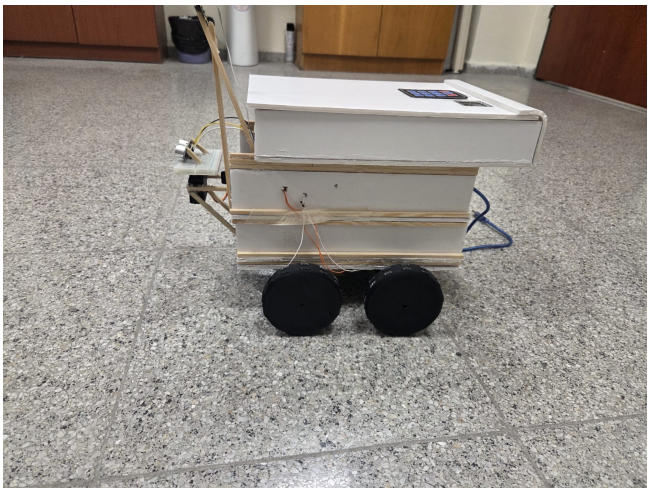


Fig. 9. Prototype and system photographs (set 9).